

helixqa_integration_sec00

HelixQA Full Integration into HelixCode System — Master Integration Plan

Executive Summary

This document presents the complete, in-depth, bluff-proof integration plan for embedding the HelixQA submodule—together with all its dependency submodules—into the HelixCode enterprise-grade distributed AI development platform. The plan is built on exhaustive analysis of three repositories (HelixCode, HelixQA, and Catalogizer), their source code, documentation, governance files, and existing QA artifacts. Every recommendation includes exact file paths, code references, and implementation details necessary for engineering execution.

Primary Objective

The integration's paramount goal is to make the HelixCode system flawless and exhaustively tested through AI-driven, heavy QA sessions that validate every client application—Web, Desktop, Mobile, CLI, and TUI—and every API service. The system must deliver on-demand screenshots for presentational purposes, captured autonomously by HelixQA during active QA sessions. Testing must guarantee real end-user usability; a passing test suite with broken features is categorically unacceptable and is treated as a critical infrastructure failure per Article XI §11.9 and CONST-035.

Key Findings from Deep Repository Analysis

HelixCode is a Go-based distributed AI development platform (v1.0.0, MIT) with six client applications: a Cobra CLI, a tvview TUI, a Fyne v2 Desktop GUI, gomobile Android/iOS bindings, and Aurora OS / Harmony OS support. Its architecture spans 34+ internal packages, REST/WebSocket/MCP APIs, and a 15 KB Makefile build system. It already contains anti-bluff rules but under the identifier CONST-017 rather than the cross-repository standard CONST-035, and critically lacks the verbatim user-mandate forensic anchor of Article XI §11.9 found in HelixQA and Catalogizer.

HelixQA is an AI-driven QA orchestration framework (Apache-2.0, 97.6 % Go) with 40+ packages including autonomous session coordination, LLM-powered navigation (ADB/Playwright/X11 executors), visual bug detection, evidence collection, and ticket generation. It already embeds CONST-035 and Article XI §11.9 in its governance files. Its autonomous session runs a four-phase lifecycle (Setup → Doc-Driven Verification → Curiosity-Driven Exploration → Report & Cleanup) across multiple platforms in parallel. However, screenshot coverage has significant gaps: no iOS capture, no Windows desktop capture, no native macOS ScreenCaptureKit integration, and no visual capture for CLI/TUI interfaces.

Catalogizer already includes HelixQA as a submodule at commit 35deb43 (Phase 27.7) but trails upstream by two full phases (0bca023, Phase 29). It maintains 41 submodules (32 vasic-digital infrastructure modules plus 9 HelixDevelopment AI/QA modules) and has a Full-QA Master Cycle defined in its constitution (Article VII). Its last audit recorded 206 PASS / 1 SKIP across 60+ test banks, yet integration gaps persist: Android/Android TV autonomous QA is

blocked, the installer-wizard and API client lack dedicated HelixQA banks, and the OCU-CUDA-Sidecar remains undeployed.

The Ten Integration Phases

The plan is organized into ten sequential phases, each with fine-grained tasks, exact file references, and anti-bluff verification criteria:

Phase	Focus	Critical Deliverable
0	Constitution & Governance	Add Article XI §11.9 to HelixCode; align CONST-017→CONST-035; cascade mandates to all 56 submodules
1	Submodule Dependencies	Register 9 submodules in <code>.gitmodules</code> ; bump Catalogizer from 35deb43→0bca023; wire <code>go.mod</code> replace directives
2	Core Integration	Create <code>internal/helixqa/wrapper</code> ; expose <code>/api/v1/qa/*</code> endpoints; register <code>helixcode qa</code> CLI commands; add TUI dashboard
3	Screenshot Pipeline	Implement <code>pkg/screenshot/</code> with 8 platform engines; deliver on-demand REST/WebSocket API; add presentational export
4	Test Coverage Matrix	Define 10 test types × 5 client categories = 50 coverage cells; mandate protocol-layer probes and visual verification
5	Catalogizer Example	Bump submodules; create 5 new test banks; validate Web/Desktop/Android/API clients with real device automation
6	Anti-Bluff Framework	Implement 4-layer architecture (protocol → functional → visual → destructive); add <code>pkg/antibluff/</code> ; design 4 synthetic user workflows
7	AI QA Orchestration	Orchestrate 4-phase autonomous sessions across all platforms; generate video-evidence reports; produce slide-deck exports
8	Enterprise UX Validation	Verify translation tool UX across 5 clients; validate 42 LLM providers; enforce 99.9 % uptime and cost controls
9	Build & Automation	Add <code>make qa-all/qa-session/qa-anti-bluff</code> targets; create session scripts; maintain NO-CI/CD constitutional compliance
10	Monitoring & Compliance	Deploy static HTML dashboards; publish anti-bluff compliance reports; establish release gates and monthly improvement reviews

Anti-Bluff Mandate — Non-Negotiable

Every phase of this plan is governed by the anti-bluff covenant. The operative rule, inherited from the user’s mandate and now codified in HelixQA’s and Catalogizer’s constitutions, states: **the bar for shipping is not “tests pass” but “users can use the feature.”** Every test, every Challenge, and every QA session result must carry positive evidence that the feature works for the end user.

A green test suite combined with a broken feature is a worse outcome than an honest red suite—it silently destroys trust in the entire system. This mandate is cascaded to every submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md as a non-negotiable, release-blocking requirement.

Resource and Safety Constraints

All test and challenge execution is strictly limited to 30–40 % of host system resources (GOMAXPROCS=2, nice -n 19, container memory caps). No CI/CD pipelines are permitted per constitutional mandate; all builds and QA sessions are triggered manually via Makefile targets or shell scripts. Host power management transitions (suspend, hibernate, reboot) are categorically forbidden and blocked by hardened systemd configuration.

Success Criteria

The integration is complete when: 1. All 56 submodules across HelixCode and Catalogizer contain CONST-035 and Article XI §11.9; 2. HelixCode exposes REST and CLI interfaces for triggering QA sessions and retrieving on-demand screenshots; 3. Screenshot engines cover all 8 client platform variants (Web responsive breakpoints, Desktop Linux/macOS/Windows, Android, iOS, CLI, TUI); 4. The 10×5 test coverage matrix is fully populated with anti-bluff verification methods; 5. Catalogizer demonstrates the integration with 5 new test banks and 206+ PASS results; 6. The anti-bluff framework detects deliberately broken features with 100 % accuracy; 7. Autonomous QA sessions run across all platforms with video evidence and ticket generation; 8. The release gate checklist passes: all tests + all challenges + all screenshots verified + all anti-bluff checks + all governance cascades confirmed.
